NOSH Computer Vision Intern Assignment

Egg Segmentation & Tracking Pipeline

Submitted by: Sourav Pati
Date: November 2026

Executive Overview

This report covers the complete pipeline I built for the NOSH CV intern assignment: segmenting eggs in still images, evaluating the segmentation approach, and tracking individual eggs across a video.

The project has four main parts:

- Generating pseudo-ground-truth masks using Meta's SAM ViT-B model
- Splitting the dataset in a way that avoids image leakage
- Building and iterating on a classical image-processing segmentation method
- Tracking multiple eggs in a video using Watershed segmentation and Hungarian matching

The final tracked video for Task 4 is available here: Google Drive link. The local output file is `outputs/video/tracked_video.mp4`, containing 1,440 frames at 24 FPS.

Task 1: Pseudo-Ground-Truth Using SAM

Since the dataset did not come with manually annotated masks, I used Meta's Segment Anything Model (SAM ViT-B) to create pseudo-ground-truth masks for all 160 images.

SAM works well as a general segmentation model, but when used automatically it often picks up background textures such as the table surface. To reduce this, I added a filtering and ranking step in `src/step1_sam_segment.py`.

For every candidate mask, I fitted an ellipse using `cv2.fitEllipse` and calculated how closely the mask area matched the ellipse area:

$$R = \min(A, A_{ellipse}) / \max(A, A_{ellipse})$$

I then ranked candidates using:

$$Score = Predicted_IoU \times Stability_Score \times R$$

After scoring, I applied greedy non-maximum suppression with `max_keep=1`, so only the most likely egg mask was kept. This helped remove table fragments and other background regions.

For close-up images where the egg occupied most of the frame, the ellipse-based constraint was too strict. I added a fallback that relaxed the geometric limits in those cases. With this adjustment, I was able to generate masks for all 160 images.

Task 2: Dataset Splitting and Classical Segmentation

Avoiding Data Leakage

A simple random train/test split would have been misleading for this dataset. Many images were Roboflow augmentations of the same original image, and some were burst-shot captures taken only seconds apart under nearly identical conditions.

To prevent leakage, I grouped related images before splitting them. In `src/step2_split.py`, I:

- Grouped images sharing the same base filename stem, such as files with `*.rf.*` naming
- Computed 8×8 perceptual difference hashes (dHash) across four orientations: original, horizontal flip, vertical flip, and diagonal flip
- Merged image pairs with a Hamming distance of 10 or less into the same cluster
- Assigned entire clusters to either the development or evaluation split

The final split used a 4:1 ratio with a fixed seed of 42. This produced 128 development images and 32 evaluation images, with no related images appearing across both sets.

Classical Segmentation Experiments

Segmenting eggs using classical methods turned out to be harder than expected. The eggs varied in colour, with both white and brown shells, and the background included off-white and wooden table surfaces. There were also specular highlights, shadows, and low-contrast regions that made simple thresholding unreliable.

I tested several approaches and recorded the results in `experiments/experiment_log.csv`.

Run	Method	Split	Mean IoU	Median IoU	Dice	Precision	Recall	Result
E01	HSV thresholding, S ≤ 60, V ≥ 140	dev	0.2991	0.0011	0.3537	0.3514	0.4096	Discarded
E02	CIELAB a* channel with Otsu	dev	0.1390	0.0001	0.1820	0.1758	0.3099	Discarded
E03	CIELAB L* channel with Otsu	dev	0.3276	0.1631	0.3881	0.3817	0.4570	Kept
E04	L* Otsu + convex hull	dev	0.3285	0.2217	0.3960	0.3688	0.5400	Kept
E05	Border Delta-E + convex hull	dev	0.3494	0.2987	0.4319	0.4784	0.4941	Selected
Final	E05 on locked evaluation set	eval	0.2825	0.0821	0.3510	0.4024	0.4015	Final result

The HSV approach failed mainly on brown eggs and light table surfaces. The a* channel also did not provide enough separation between the egg and background.

The L* channel performed better because it captured diffuse reflections from the eggshell. Adding a convex hull improved recall by filling in glare holes inside the egg mask.

The best method was E05, which used a border-based CIELAB Delta-E distance prior, followed by Otsu thresholding, morphological filtering, and convex hull correction. It handled both white and brown eggs more consistently than the earlier approaches.

Task 3: Evaluation and Failure Analysis

I evaluated the final E05 pipeline on the locked evaluation split, which contained 31 valid unseen images.

Metric	Mean	Median	25th percentile	75th percentile	Maximum
IoU	0.2825	0.0821	0.0000	0.5763	0.8289
Dice coefficient	0.3510	0.1517	0.0000	0.7312	0.9065
Precision	0.4024	0.2610	0.0000	0.8875	1.0000
Recall	0.4015	0.5219	0.0000	0.7643	1.0000
Pixel accuracy	0.6584	0.6847	0.5145	0.8432	0.9495

The results show that the method works reasonably well on many images, but still fails badly on certain difficult cases. The low median IoU indicates that a meaningful portion of the evaluation images have very poor segmentation.

Main Failure Cases

Small or distant eggs. For images such as `not_damaged_118` and `not_damaged_27`, the egg occupies less than 5% of the frame. Lighting gradients and soft table shadows can create a connected foreground region larger than the egg itself. Since the pipeline keeps only the largest connected component, it sometimes selects the shadow and loses the egg completely, resulting in an IoU of zero.

Extreme close-ups. In images such as `not_damaged_6`, the egg extends beyond all four sides of the frame. The border-sampling region, which is meant to capture background table pixels, ends up sampling eggshell instead. This causes the Delta-E distance map to invert, and the algorithm segments the darker table corners rather than the egg.

Speed Advantage

Although the classical pipeline is less semantically powerful than SAM, it is much faster. It takes roughly 37 ms per image on CPU, while SAM ViT-B takes about 12.4 seconds per image on GPU. That is approximately a 330× speed improvement, which makes the classical approach more practical for real-time or edge-device applications.

Task 4: Multi-Object Egg Tracking

For the video task, the goal was to track individual eggs separately as they moved through a YouTube video sequence. I implemented the tracking pipeline in `src/step5_video_track.py`.

Instance Separation

When eggs touch or overlap, a simple connected-component approach would treat them as one object. To separate them, I computed the distance transform on the binary mask and applied OpenCV's Watershed algorithm. This helped split adjacent egg instances more cleanly.

Filtering Non-Egg Objects

Not every detected blob corresponds to an egg. The video also contains tablecloth folds, sleeves, and channel logos. I filtered candidate blobs using `is_egg_candidate`, based on:

- Area between 800 and 25,000 pixels
- Aspect ratio between 0.65 and 1.45
- Solidity of at least 0.80, calculated as blob area divided by convex hull area
- Rejection of blobs touching the image corners

Tracking Across Frames

For each valid egg detection, I extracted its centroid and matched detections between consecutive frames using the Hungarian algorithm (`linear_sum_assignment`) on a Euclidean distance matrix.

I also used a simple constant-velocity motion model:

$$v_t = 0.5\, v_{(t-1)} + 0.5\, \Delta c$$

To handle short occlusions, the tracker allows an object to coast for up to 15 frames before the track is terminated.

The final processed video contains 1,440 frames, runs for 60 seconds at 24 FPS, and has a resolution of 640×360. It is saved as `outputs/video/tracked_video.mp4`.

AI Tools Disclosure

As required by the submission policy, I used AI coding assistants, including Gemini and Claude through the Antigravity IDE, for some implementation support. Specifically, they helped with CLI argument parsing, boilerplate scripting, Matplotlib figure layouts, and Markdown documentation structure.

All core technical decisions, implementation choices, debugging, and validation were done by me. This includes the perceptual dHash-based leakage prevention, the border-prior CIELAB Delta-E segmentation method, convex hull repair, Watershed instance separation, and Hungarian-based motion tracking.